

# CS4800: Designing Sustainable ICT Systems

## Lecture 3: Sustainable Software

**September 18, 2025**

Lecturer: Przemysław Pawełczak

Authors: Alexandru Postu, Radosław Majer

[CS4800-EWI@tudelft.nl](mailto:CS4800-EWI@tudelft.nl)

# What will you learn in this lecture?

## Week 1.3: Sustainable Software

- Introduction to Sustainable Software
- Measuring software sustainability
- Sustainable design and development practices
- Contemporary sustainable solutions for various platforms, and their consequences for a software engineer

# What is sustainable software? How do we achieve it?

# Basics: definitions

- **Sustainable software:** a software that emits the *least amount of carbon possible* while maintaining the same desired functionality
- **How can this be achieved?**
  - Through energy, carbon and hardware efficiency

# Basics: principles

According to the [Green Software Foundation](#), there exist six base principles:

1. **Carbon Efficiency:** Software emits the least amount of carbon possible
  - Optimize software to use less resources
  - *Example:* not storing redundant data helps minimize the store usage of data centers
2. **Energy Efficiency:** Software uses the least amount of energy possible
  - Design software solutions to be more efficient, and thus consume less energy.
  - *Example:* efficient algorithms or data structures
3. **Carbon Awareness:** Do more when the electricity is cleaner and do less when the electricity is dirtier
  - Schedule your software workload jobs according to the energy mix in a grid ([example](#))
  - The [UK's Carbon Intensity API](#) forecasts the carbon intensity 96+ hours ahead

# Basics: principles

4. **Hardware Efficiency:** Use the least amount of embodied carbon possible
  - Maximize the lifespan of hardware
  - Use the current resources to their maximum potential
5. **Measurement:** What you can't measure, you can't improve
  - Use measurement tools to perform experiments on your software
  - Come up with concrete measurement metrics and methods
6. **Climate Commitments:** Understand the exact mechanism of carbon reduction
  - Prioritize actually reducing emissions

**Recap from the last lecture: What does *social sustainability* mean? What does it mean that a society is *sustainable*?**

**A society is sustainable in social terms if people do not face structural obstacles in key areas.**



# Back to software: *how do we measure software sustainability?*

# Measurements: definitions

- **Energy efficiency:** granular, referring to the energy consumption of running an application only.
- **Carbon efficiency:** encompasses the entire development cycle of an application.
  - *Example:* the production of necessary hardware to host the data on.

# Measurements: metrics

## Energy

- Most commonly measured in Joules (J) or kilowatts-hour (kWh).
- Can be measured with the help of energy profilers

## Power

- The amount of energy used per unit of time
- Commonly measured in Watts
- Can also be measured with energy profilers

# Measurements: metrics

## Average Power

- This is used when power is not consistent over time
- Average power divided over the length of a time interval

$$E = P_{\text{average}} \Delta t$$

## Energy Delayed Product

- While some applications do not consume too much energy, they might be very slow
- The EDP favors applications that minimize both energy consumption and runtime

$$\text{EDP} = Et = \Delta P t^2$$

# Measurements: hardware

## Watt-meter

- An simple instrument measuring the electric power in watts of any given circuit, in this case a power used by a device connected to a power socket
- Averages the

$$v(t)i(t)$$

over a time interval

- You can plug a Watt-meter into a power socket
- This measurement is most accurate, but offers no granularity - you can only measure the power consumption of the entire system, not a specific application



[Example watt-meter](#)

# Measurements: hardware

## High voltage power monitor

- A commercial device sold by Monsoon Solutions Inc.
- Can be used for mobile devices, tablets, laptops
- Connects to the device through USB, and to the power source through a special cable
- Provides an extensive API for data collection and analysis
- This measurement is most accurate, but offers no granularity - you can only measure the power consumption of the entire system, not a specific application



[High voltage power monitor](#)

# Measurements: software

## Energy profilers

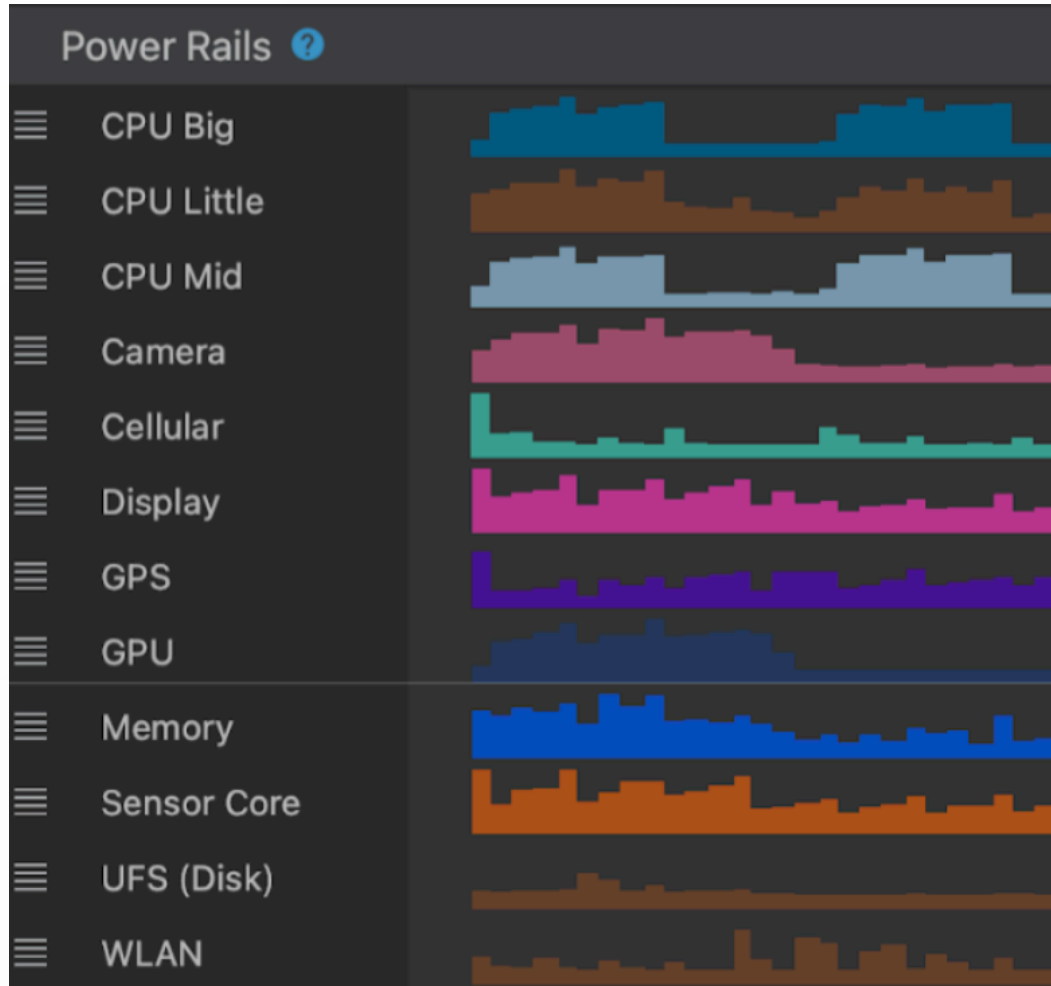
- They monitor the hardware resources of an app - CPU, memory, disk, network
- They provide more granular information - which part of the app uses which hardware resource
- However, they provide an **estimate** of energy/power used - there could always be inaccuracies

# Profilers: example

- [CodeCarbon](#): Popular tool that measures power consumption, but also carbon intensity, which is useful for AI solutions. Available for Python applications

```
from codecarbon import EmissionsTracker
tracker = EmissionsTracker() # Initialize the tracker
tracker.start() # Start tracking
total = 0
for i in range(1, 10000000):
    total += i
emissions = tracker.stop() # Stop tracking and get emissions
print(f"Estimated CO2 emissions: {emissions} kg")
```





[Power Profiler in Android Studio](#)

## Profilers: example

- [Android Studio Power Profiler](#): Popular tool that measures power consumption, but also carbon intensity, which is useful for AI solutions; Available for Python applications
- How do you use it? When using the Android Studio IDE, you can access the profiler while debugging an app on a connected device or a virtual device (emulator)

**Much** more examples in the lecture notes!

# Methodology

- A methodology to assess software sustainability is necessary
- This methodology must:
  - minimize confounding variables
  - be reproducible and give confidence in the measured outcomes
- To minimize confounding variables:
  - Switch off unnecessary processes
  - Disconnect external hardware
  - Ensure system settings stay consistent
  - Keep hardware cool
  - If performing an experiment with multiple setups, shuffle their order

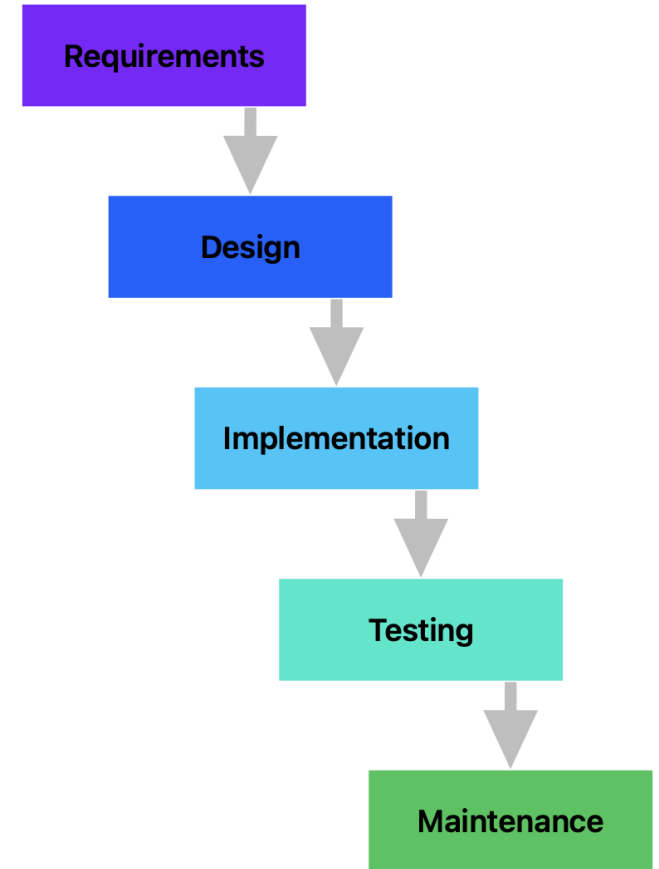
# Methodology: experimental procedure

- Repeat your experiments multiple times, to ensure reproducibility
- Visualize the different results through graphs (e.g: boxplots)
- You might have outliers, detect this through:
  - visual inspection
  - statistical tests (e.g: [Shapiro Wilk](#) - tests whether the data is normally distributed)

**What sustainable software development practices are there? How can we classify them?**

# Practices: classification

Recall the waterfall model. While in modern software engineering it is not that relevant anymore, we will use it to identify possible improvements in the process.



Waterfall model

# Requirements

- The energy efficiency is a non-functional requirement
- Usual scenario: we want the software to use hardware resources more efficiently

## Four main requirement types

- computational efficiency
- data management - reducing I/O operations to storage
- data communication - network use
- energy consumption awareness - providing energy information to users

# Requirements: exercise

*Can you classify these requirements into the four categories?*

- The system must provide the battery information in the settings menu, containing the current battery level, estimated time remaining, battery health status, and power consumption details by specific apps.
- The algorithm must be optimized to a time complexity of  $O(n \log n)$ .
- When in the gallery view, only the thumbnails of images should be loaded initially, with full-resolution images loaded on demand.
- The messages containing media files must be compressed before transmission.

***computational efficiency | data management | data communication | energy consumption awareness***

# Requirements: exercise

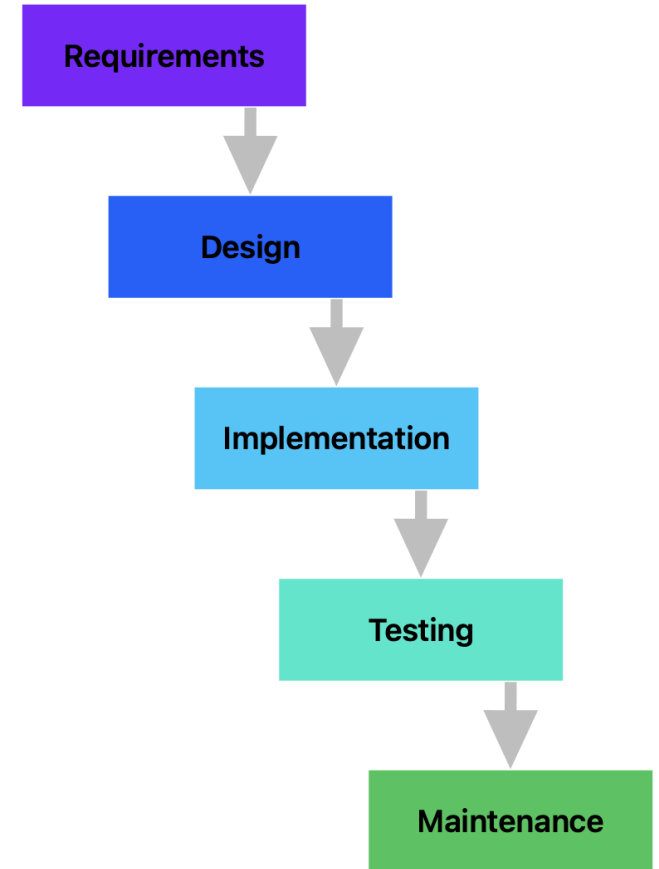
*Can you classify these requirements into the four categories?*

- The system must provide the battery information in the settings menu, containing the current battery level, estimated time remaining, battery health status, and power consumption details by specific apps. **[energy consumption awareness]**
- The algorithm must be optimized to a time complexity of  $O(n \log n)$ . **[computational efficiency]**
- When in the gallery view, only the thumbnails of images should be loaded initially, with full-resolution images loaded on demand. **[data management]**
- The messages containing media files must be compressed before transmission. **[data communication]**



# Practices: classification

Recall the waterfall model. While in modern software engineering it is not that relevant anymore, we will use it to identify possible improvements in the process.



Waterfall model

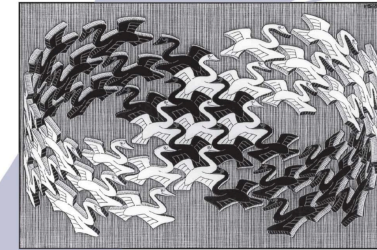
# Design

- Most contemporary software infrastructure - written in an object oriented manner → conventional design patterns
- **Design patterns:** reusable solutions to common problems in software design
- Design patterns are about the use and arrangement of classes and objects, and the relationships between them
- Can we qualify the energy consumption according to the design decision?

## Design Patterns

Elements of Reusable  
Object-Oriented Software

Erich Gamma  
Richard Helm  
Ralph Johnson  
John Vlissides



Cover art © 1994 M.C. Escher / Cordon Art - Baarn - Holland. All rights reserved.

Foreword by Grady Booch



Fundamental Design Patterns  
Textbook

# Design: example

```
1. class RealImage {
2.     public:
3.     RealImage(FILE *f) { ... }
4.     ~RealImage() { ... }
5.     void draw() { ... }
6. };
7. class Image {
8.     RealImage *realImage;
9.     FILE *file;
10.    public:
11.    Image(FILE *f) {
12.        realImage = 0;
13.        file = f;
14.    }
15.    ~Image() { ... }
16.    void draw() {
17.        if (!realImage)
18.            realImage =
19.                new RealImage(file);
20.        realImage->draw();
21.    }
22. };
(a)
```

```
1. class Image {
2.     public:
3.     Image(FILE *f) { ... }
4.     ~Image() { ... }
5.     void draw() { ... }
6. };
(b)
```

```
1. int main(int argc, char** argv) {
2.     for( ... ) {
3.         Image i1 = new Image("...")
4.         Image i2 = new Image("...")
5.         Image i3 = new Image("...")
6.         i1.draw();
7.         i3.draw();
8.         i1.draw();
9.     }
10. }
```

(c)

[proxy pattern \(a\) before \(b\) after applying and \(c\) driver code](#)

# Design: recent study

- **RQ: Is it possible to predict the impact on energy usage of applying a design pattern by examining design-level artifacts?**
- (...) a reliable prediction about the impact of applying a design pattern can not be made by considering only high-level design artifacts.
- (...) impact on energy usage appears to be connected with the change in number of objects and number of messages passed between objects. (...) There are exceptions to this general trend

# Design: recent study

- Methodology: the code before and after applying a design pattern was executed for the number of iterations
- the energy consumption (in Jules) was measured with a hardware device
- the number of objects instantiated and messages passed between objects was recorded

| Design pattern   | # Iterations | Energy usage |           |                |                              |          | # Objects |       | # Messages |       |
|------------------|--------------|--------------|-----------|----------------|------------------------------|----------|-----------|-------|------------|-------|
|                  |              | Before (J)   | After (J) | Difference (J) | Difference per iteration (J) | % Change | Before    | After | Before     | After |
| Creational       |              |              |           |                |                              |          |           |       |            |       |
| Abstract factory | 500          | 87.78        | 106.69    | 18.91          | 0.0378                       | 21.55    | 11        | 13    | 7          | 12    |
| Builder          | 750          | 111.75       | 113.08    | 1.33           | 0.0018                       | 1.19     | 3         | 6     | 18         | 32    |
| Factory method   | 500          | 118.13       | 118.06    | -0.08          | -0.0002                      | -0.07    | 3         | 3     | 1          | 2     |
| Prototype        | 750          | 99.60        | 98.68     | -0.93          | -0.0012                      | -0.93    | 7         | 7     | 3          | 9     |
| Singleton        | 250          | 98.70        | 99.11     | 0.42           | 0.0017                       | 0.42     | 2         | 2     | 7          | 12    |
| Structural       |              |              |           |                |                              |          |           |       |            |       |
| Bridge           | 35           | 99.78        | 99.54     | -0.24          | -0.0070                      | -0.24    | 6         | 11    | 3          | 6     |
| Composite        | 175          | 97.15        | 102.14    | 4.99           | 0.0285                       | 5.14     | 17        | 19    | 10         | 11    |
| Decorator        | 115          | 13.92        | 113.13    | 99.21          | 0.8627                       | 712.89   | 14        | 24    | 14         | 15    |
| Flyweight        | 500          | 92.89        | 38.94     | -53.95         | -0.1079                      | -58.08   | 60        | 6     | 60         | 60    |
| Proxy            | 500          | 104.33       | 66.28     | -38.05         | -0.0761                      | -36.47   | 5         | 2     | 3          | 6     |
| Behavioral       |              |              |           |                |                              |          |           |       |            |       |
| Command          | 750          | 98.32        | 96.53     | -1.79          | -0.0024                      | -1.82    | 7         | 7     | 29         | 29    |
| Mediator         | 250          | 120.55       | 109.22    | -11.53         | -0.0461                      | -9.56    | 4         | 5     | 23         | 26    |
| Observer         | 400          | 61.62        | 99.95     | 38.33          | 0.0958                       | 62.20    | 3         | 7     | 4          | 8     |
| Strategy         | 500          | 115.73       | 115.52    | -0.21          | -0.0004                      | -0.18    | 4         | 3     | 9          | 12    |
| Visitor          | 90           | 104.89       | 97.04     | -7.86          | -0.0873                      | -7.49    | 10        | 14    | 16         | 26    |

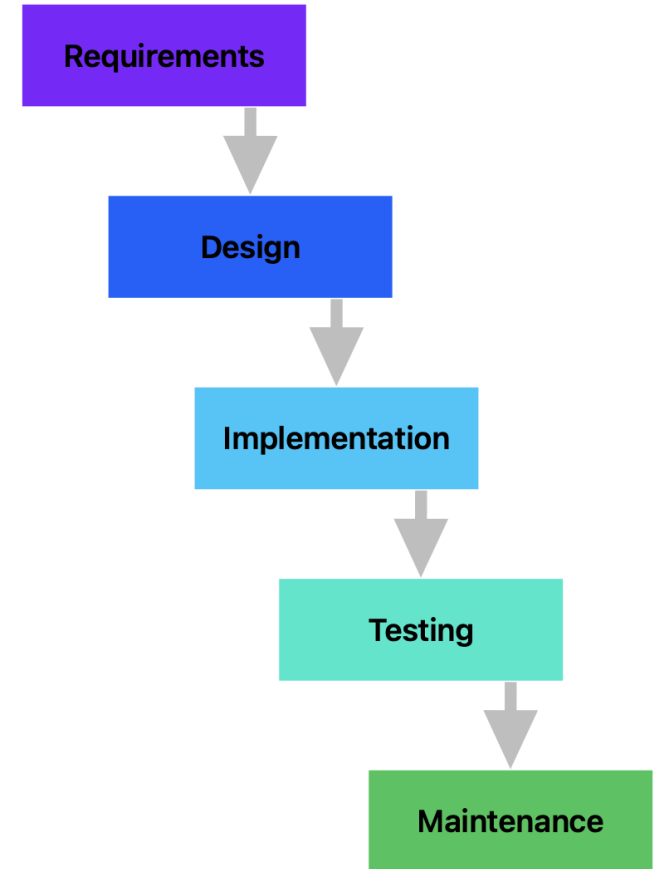
## [Design Patterns Study](#)

# Design: green design patterns

- Flyweight: sharing data between similar object
  - *Example*: in a text editor, instead of creating a new object for each character, share the same object for all instances of the same character
- Proxy: providing a placeholder for another object to control access or add functionality
  - *Example*: load high-resolution images in the gallery only when requested, load thumbnails otherwise
- Observer: maintaining a list of observers and notifying them of events
  - *Example*: a weather application notifying users of changes in weather conditions

# Practices: classification

Recall the waterfall model. While in modern software engineering it is not that relevant anymore, we will use it to identify possible improvements in the process.



Waterfall model

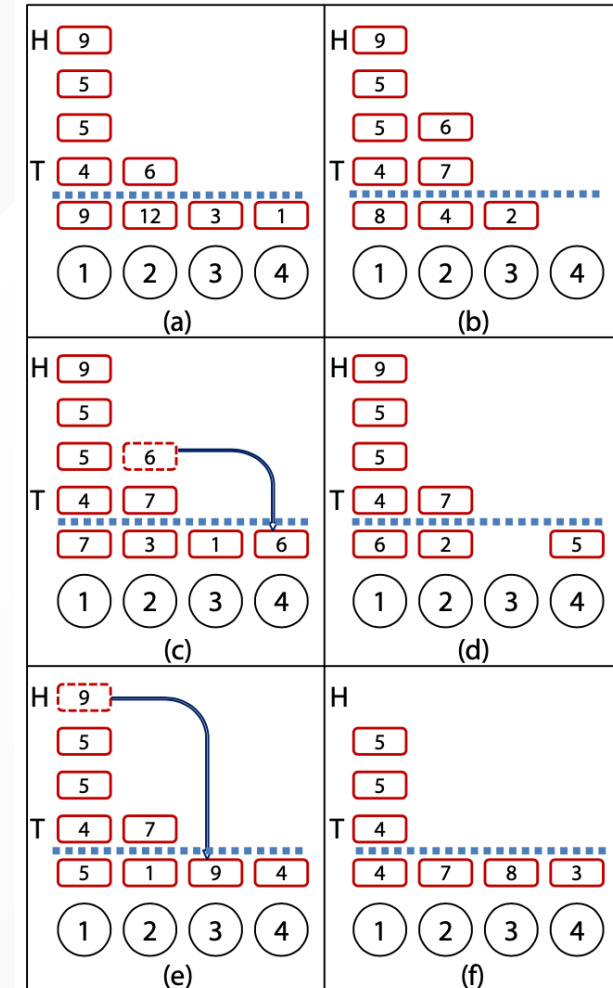
# Implementation: work stealing

- **Work stealing:** thread management strategy
  - With its root in Cilk language, work stealing is widely available in industry-strength C/C++/C#-based language frameworks such as Intel TBB, Intel Cilk Plus, and Microsoft .NET framework
  - The core idea of work stealing made its way into mainstream languages such as Java, X10, Haskell, and Scala.



# Implementation: work stealing

- A program consists of *workers* threads executing on a host CPU
- Each *worker* maintains a dequeue of tasks, if the queue is empty, the worker *steals* a task from a *victim*
- **Work-first principle:** when spawning a new thread, execute the thread and enqueue the further instructions (put them as the most immediate)
- When a queue of a worker has been emptied, it retrieves a task from the *head* of the other worker's queue, so the least immediate task



Work

stealing.

# Implementation: Hermes

- [Hermes](#): a step toward energy efficiency for *workstealing* runtimes
- Framework is tempo-enabled - utilizes the possibility of adjusting CPU frequency
- Hermes introduces the following optimizations:
  - **Workpath-Sensitive Tempo Control** - at the beginning of thief-victim relationship, thief's tempo is slowed down (thief procrastination)
  - once the relationship ends because victim has run out of tasks, thief's tempo is again raised up (immediacy relay)
  - **Workload-Sensitive Tempo Control** - worker's tempo is dependent on the number of tasks to be completed

# Implementation: loop unrolling

- Replacing a loop with multiple copies of the loop body - many sequential statements instead of a loop with a small number of iterations.
- Loop unrolling happens on the compilation level
- Increased instruction-level parallelism - the processor can execute further instructions while waiting for the previous ones to complete
- Reduces the runtime computations of the loop header
- Often used in encryption and decryption methods, which contain multiple nested loops iterating from 0 to 9, which can be unrolled

# Implementation: loop unrolling

```
for (int x = 0; x < 10; x++) {  
    remove(x);  
}
```

```
remove(0);  
remove(1);  
remove(2);  
//...  
remove(9);
```

# Implementation: macros vs functions

From the developer perspective, macros and functions are similar - they both allow code reuse. However, they are different in terms of compilation and execution.

## Macros

- Replaced by the preprocessor before compilation - the call is replaced by the actual code
- For small operations, macros can be more efficient in terms of time, and therefore cause less energy use

## Functions

- Compiled and executed at runtime - the call is replaced by a jump to the function code
- For large operations, functions are preferred to macros, as they avoid code bloat - replacing every call with actual code may cause increased memory usage

# Implementation: code generating AI

- Tools such as GitHub Copilot, ChatGPT, and others are becoming increasingly popular
- They can help developers write more efficient code
- However, the cost of training and running these models is very high
- Multiple studies [example](#) suggest that these tools waste a lot of computational power through redundant, unused queries (cancelled queries, rejected suggestions, etc.)
- **You will learn more about green AI in the future lectures**

**Are all languages equally efficient? Can we simply build all software in the most efficient language?**

# Language efficiency: recent study

- **RQ: Can we compare the energy efficiency of software languages? Can we automatically decide what is the best software language considering energy, time, and memory usage?**
- Compiled languages tend to be, as expected, the fastest and most energy efficient ones
- If the developer is only concerned with execution time and energy consumption, then (...) it is almost always possible to choose the best language. (...) If memory is also a concern, it is no longer possible to automatically decide for a single language.
- Reminder: the energy does not depend solely on execution time



# Language efficiency: recent study

- Methodology: 27 programming languages have been tested on 10 benchmark problems
- For each problem, the most optimal implementations in different languages were collected
- Information about average power was collected and with Intel's Running Average Power Limit (RAPL) tool
- The table on the right presents the mean of the normalized average energy values for each language

|                | Energy |                | Time  |                | Mb    |
|----------------|--------|----------------|-------|----------------|-------|
| (c) C          | 1.00   | (c) C          | 1.00  | (c) Pascal     | 1.00  |
| (c) Rust       | 1.03   | (c) Rust       | 1.04  | (c) Go         | 1.05  |
| (c) C++        | 1.34   | (c) C++        | 1.56  | (c) C          | 1.17  |
| (c) Ada        | 1.70   | (c) Ada        | 1.85  | (c) Fortran    | 1.24  |
| (v) Java       | 1.98   | (v) Java       | 1.89  | (c) C++        | 1.34  |
| (c) Pascal     | 2.14   | (c) Chapel     | 2.14  | (c) Ada        | 1.47  |
| (c) Chapel     | 2.18   | (c) Go         | 2.83  | (c) Rust       | 1.54  |
| (v) Lisp       | 2.27   | (c) Pascal     | 3.02  | (v) Lisp       | 1.92  |
| (c) Ocaml      | 2.40   | (c) Ocaml      | 3.09  | (c) Haskell    | 2.45  |
| (c) Fortran    | 2.52   | (v) C#         | 3.14  | (i) PHP        | 2.57  |
| (c) Swift      | 2.79   | (v) Lisp       | 3.40  | (c) Swift      | 2.71  |
| (c) Haskell    | 3.10   | (c) Haskell    | 3.55  | (i) Python     | 2.80  |
| (v) C#         | 3.14   | (c) Swift      | 4.20  | (c) Ocaml      | 2.82  |
| (c) Go         | 3.23   | (c) Fortran    | 4.20  | (v) C#         | 2.85  |
| (i) Dart       | 3.83   | (v) F#         | 6.30  | (i) Hack       | 3.34  |
| (v) F#         | 4.13   | (i) JavaScript | 6.52  | (v) Racket     | 3.52  |
| (i) JavaScript | 4.45   | (i) Dart       | 6.67  | (i) Ruby       | 3.97  |
| (v) Racket     | 7.91   | (v) Racket     | 11.27 | (c) Chapel     | 4.00  |
| (i) TypeScript | 21.50  | (i) Hack       | 26.99 | (v) F#         | 4.25  |
| (i) Hack       | 24.02  | (i) PHP        | 27.64 | (i) JavaScript | 4.59  |
| (i) PHP        | 29.30  | (v) Erlang     | 36.71 | (i) TypeScript | 4.69  |
| (v) Erlang     | 42.23  | (i) Jruby      | 43.44 | (v) Java       | 6.01  |
| (i) Lua        | 45.98  | (i) TypeScript | 46.20 | (i) Perl       | 6.62  |
| (i) Jruby      | 46.54  | (i) Ruby       | 59.34 | (i) Lua        | 6.72  |
| (i) Ruby       | 69.91  | (i) Perl       | 65.79 | (v) Erlang     | 7.20  |
| (i) Python     | 75.88  | (i) Python     | 71.90 | (i) Dart       | 8.64  |
| (i) Perl       | 79.58  | (i) Lua        | 82.91 | (i) Jruby      | 19.84 |

[Language Efficiency Study - normalized results  
per language](#)

# Platform efficiency: task scheduling

- The power consumption of a processor grows nearly-exponentially with frequency growth
- Modern processors - multiple cores with multiple frequencies
- Task scheduling - deciding which task runs on which core at what frequency - can be optimized for energy efficiency

## Examples of use

- [Energy Aware Scheduling \(EAS\)](#) - Linux kernel module; for heterogeneous multi-core processors
- [Windows EcoQoS](#) - quality of service framework for energy efficiency; allows developers to assign energy priorities to threads

```

void backgroundTask() {
    // Request EcoQoS for energy-efficient execution
    SetThreadInformation(
        GetCurrentThread(), // thread to be modified
        ThreadPowerThrottling, // type of information to set, a constant
        new POWER_THROTTLING_STATE{
            sizeof(POWER_THROTTLING_STATE), // size of the structure
            POWER_THROTTLING_EXECUTION_SPEED, // speed
            TRUE // state mask, enable
        }, // a pointer to a new state
        sizeof(POWER_THROTTLING_STATE) // size of the structure
    );

    // running efficiently
    // ...
}

int main() {
    // spawn the energy-efficient thread
    std::thread t(backgroundTask);
    t.join();
    return 0;
}

```

# Platform efficiency: just-in-time compilation

- **Java Virtual Machine (JVM):** a platform for running Java applications
- JVM uses *just-in-time compilation* extensively
- **Static compilation:** the code is compiled prior to execution, and the same binary is executed every time
- **Dynamic compilation / Just-in-time compilation:** a compilation approach that is essentially used to gain performance during the program execution
  - In **just-in-time compilation** the code is compiled during execution of a program, the compiler gathers data about the code, how it behaves, at run time - rather than prior to execution, allowing for optimizations based on the actual execution profile of the program
  - The optimizations typically include identifying frequently executed code paths (hot spots) and optimizing them for better performance

# A lot of JIT optimizations exist

## Optimizations in HotSpot JVM

- compiler tactics
  - delayed compilation
  - tiered compilation
  - on-stack replacement
  - delayed reoptimization
  - program dependence graph rep.
  - static single assignment rep.
- proof-based techniques
  - exact type inference
  - memory value inference
  - memory value tracking
  - constant folding
  - reassociation
  - operator strength reduction
  - null check elimination
  - type test strength reduction
  - type test elimination
  - algebraic simplification
  - common subexpression elimination
  - integer range typing
- flow-sensitive rewrites
  - conditional constant propagation
  - dominating test detection
  - flow-carried type narrowing
  - dead code elimination
- language-specific techniques
  - class hierarchy analysis
  - devirtualization
  - symbolic constant propagation
  - autobox elimination
  - escape analysis
  - lock elision
  - lock fusion
  - de-reflection
- speculative (profile-based) techniques
  - optimistic nullness assertions
  - optimistic type assertions
  - optimistic type strengthening
  - optimistic array length strengthening
  - untaken branch pruning
  - optimistic N-morphic inlining
  - branch frequency prediction
  - call frequency prediction
- memory and placement transformation
  - expression hoisting
  - expression sinking
  - redundant store elimination
  - adjacent store fusion
  - card-mark elimination
  - merge-point splitting
- loop transformations
  - loop unrolling
  - loop peeling
  - safepoint elimination
  - iteration range splitting
  - range check elimination
  - loop vectorization
- global code shaping
  - inlining (graph integration)
  - global code motion
  - heat-based code layout
  - switch balancing
  - throw inlining
- control flow graph transformation
  - local code scheduling
  - local code bundling
  - delay slot filling
  - graph-coloring register allocation
  - linear scan register allocation
  - live range splitting
  - copy coalescing
  - constant splitting
  - copy removal
  - address mode matching
  - instruction peepholing
  - DFA-based code generator

[JIT JVM optimizations](#)

# Maintenance

- Once the software is deployed, it must be maintained
- [Maintenance activities](#) involve correcting faults, improving performance or other attributes, and adapting the product to a modified environment
- Maintenance cost time and engineering resources
  - *Example:* when a system is being continuously developed, a CI/CD pipeline is run often to run the tests and deploy the new version in all environments, the pipeline can also be optimized for energy efficiency
- Practices to reduce maintenance cost include:
  - Proper documentation
  - Consistency in testing
  - Eliminating dead code and repeated code

# Recap

- There are six major principles of sustainable software engineering.
- We can measure the energy and carbon efficiency of software solutions with the help of energy profilers.
- We can classify sustainable software development practices into the stages of the waterfall model, for each of them there exist multiple practices.
- We can quantify the efficiency of programming languages, but the choice of the best language is not straightforward.
- The platform on which the software is executed can also be optimized for energy efficiency.

# Exam(ple) questions

1. Is loop unrolling is a good optimization for large (like 1000000) number of iterations?



# Exam(ple) questions

1. Is loop unrolling is a good optimization for large (like 1000000) number of iterations?

**No**

- Reminder: loop unrolling is a good optimization for small number of iterations, as it does not cause code bloat and increases instruction-level parallelism.

# Exam(ple) questions

2. **How does the flyweight pattern aims to reduce memory usage?**
- a. By using a single instance for all objects
  - b. By compressing data before storage
  - c. By sharing common data between similar objects
  - d. By avoiding the use of objects altogether

## 2. How does the flyweight pattern aims to reduce memory usage?

- a. By using a single instance for all objects
- b. By compressing data before storage
- c. By sharing common data between similar objects**
- d. By avoiding the use of objects altogether

# Feedback

- This is the first edition of the course - we need your feedback!
- Feedback on Lecture 3 slides/notes:



# Thank you for your attention!



## Coming tomorrow: Guest lecture with Wilco Burggraaf

- Wilco Burggraaf is a Principal Lead of Green Software Engineering at HighTech Innovators
- He focuses on research, training, and practical solutions to drive purposeful software engineering while fostering collaboration and diversity
- He will share his experience and insights on sustainable software engineering in practice
- The link to the lecture will be shared on Brightspace, stay tuned!

# Thank you for your attention!

## Next lecture: Sustainable Cloud Computing

**Thursday, September 25, 15:45-17:30, EEMCS Hall F**

- **Lecture 4 summary**

- Data center architecture: computing, cooling, power
- Environmental footprint of cloud computing, measurement methods
- Industry trends: compute growth, efficiency gains, energy
- Prospective solutions: cloud strategy, demand response, cooling technologies, waste heat reuse